Code559

Front-End Developer Course

MODULE 7

Functions

Name a set of instructions once, hand it inputs, and get a value back you can test.

Free lesson: 559solutions.com/code559/module-07-functions/

In this packet

1. **The lesson**
2. **Project: A tested helpers.js**
3. **Finishing work in Git**
4. **Troubleshooting: six common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to define, call, test and return values from functions, including how to write one from a one-sentence description. It's how working developers check their code: a function that hands back a value can be tested with made-up inputs before any web page exists.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What is a function? 5
  - 2. What is the difference between defining and calling? 5
  - 3. What is a parameter, and what is an argument? 6
  - 4. What does `return` do, and why is `console.log()` not the same? 7
  - 5. What is local scope? 8
  - 6. How do you test a function by itself? 9
  - 7. How do you write a function from a one-sentence description? 10
  - 8. What is the difference between `showMessage` and `showMessage()`? 10
  - 9. How does a function say "there is no answer"? 12
- Exercise 13
- Common misconception 15
- Summary 15
- Check your understanding 15
- Before next module 15
- Go further 16
- What to keep 16

Part 2: Project: A tested helpers.js 17

Part 3: Finishing work in Git 27

Part 4: Troubleshooting: six common problems 30

Part 5: Self-assessment 34

Part 6: Answers 36

About 559 Solutions 42

© 2026 559 Solutions LLC

PART 1

The lesson

Name a set of instructions once, hand it inputs, and get a value back you can test.

Time: about 2 hours (estimate)

In this module you will learn:

- What a function is: a named, reusable set of instructions
- The difference between defining a function and calling it
- The difference between a parameter and an argument
- What `return` does, and why `console.log()` is not the same thing
- Basic local scope: why a variable made inside a function cannot be seen outside it
- How to test a function by itself, with several inputs, in the Console and in Node
- A five-step method for writing a function from a one-sentence description
- The difference between a function reference (`showMessage`) and a function call (`showMessage()`), and why event listeners in Module 9 need the first one
- How to return `null` when there is no answer, and check for it with `=== null`

Why this matters

So far your JavaScript runs top to bottom, once. Every time you want the same work done again, you would have to type it again. A function fixes that. You write the steps once, give them a name, and run them whenever you want, with different inputs each time.

There is a second reason, and it matters more. A function that **hands a value back** can be tested by itself. You can call it with three made-up inputs and check three answers before any web page exists. That is how working developers check their code, and it is the shape of almost every practice question and interview task you will meet: "write a function that takes this and returns that."

This module stays in the Console and in Node. There is no web page yet. In Module 9 you connect functions to buttons, and the last concept here, reference versus call, is the bridge that makes that work.

Before you start

1. Finish Module 6. You should be able to use `const` and `let`, compare values with `===` and `<`, and write an `if` statement.
2. In your course folder (for example `Documents\code559`), create a throwaway folder called `practice-07`. Nothing in it is published. Open it in VS Code with **File → Open Folder...**.
3. Open a terminal in VS Code with **Terminal → New Terminal**. Type `node --version`. Any `v` followed by numbers means Node is ready.

© 2026 559 Solutions LLC

4. Open Edge or Chrome. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Anything you type there runs at once and nothing is saved. Keep it open as your scratch pad.

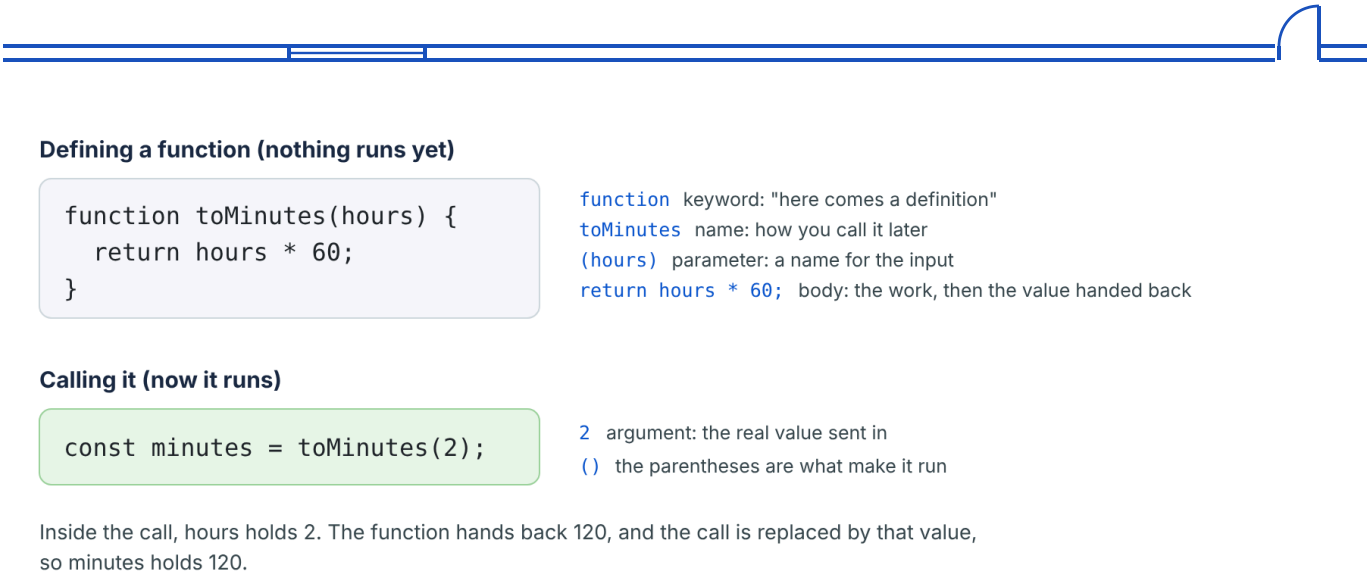
> **Checkpoint.** You have an empty `practice-07` folder open in VS Code, a terminal whose prompt ends in `practice-07>`, and a browser Console you can type into.

Core concepts

1. What is a function?

A function is a named, reusable set of instructions. You write the instructions once. After that, the name runs them.

A function definition has four parts. The keyword `function` says a definition is coming. The **name** is how you run it later. The **parameters** in parentheses are names for the inputs. The **body** in curly braces is the work. The line that starts with `return` hands a value back.



Top: the definition. Bottom: a call. The parentheses after the name are what make it run.

Point at each part of `function toMinutes(hours) { return hours * 60; }` and name it to yourself, out loud: keyword, name, parameter, body.[1]

2. What is the difference between defining and calling?

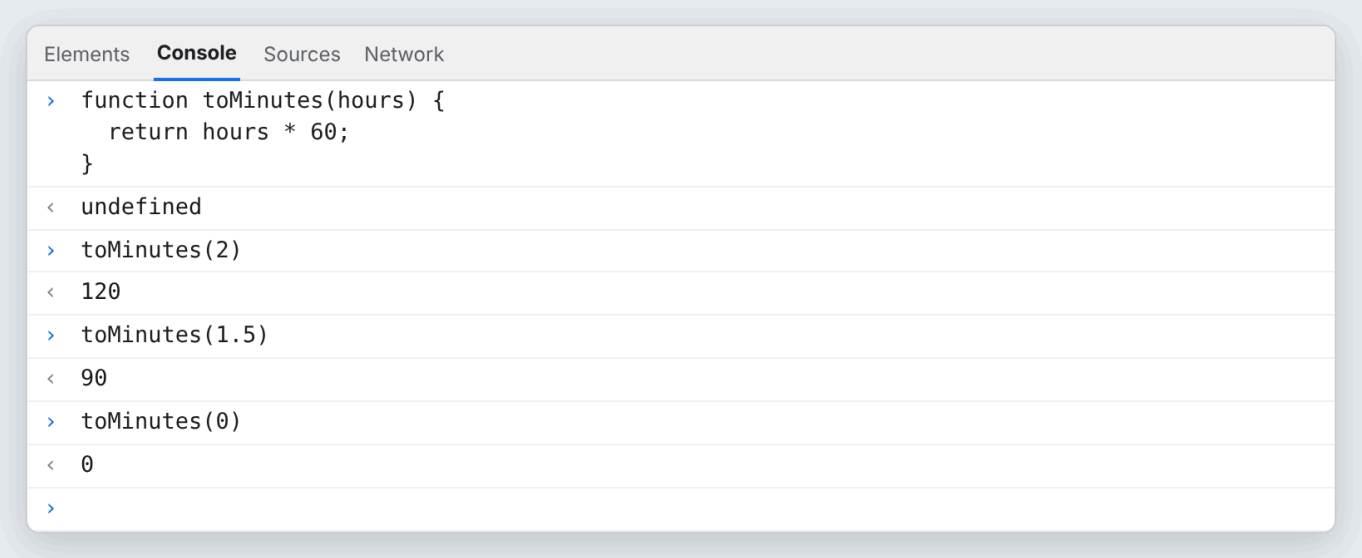
Writing a function down and running it are two separate steps. MDN's *Functions* guide puts it this way: "Defining a function does not execute it."

© 2026 559 Solutions LLC

When you define `toMinutes`, JavaScript stores the instructions under that name and does nothing else. When you **call** it, by writing the name followed by parentheses, `toMinutes(2)`, the body runs. You can call one definition as many times as you like.

Typed into the Console for the picture below, in this order:

```
function toMinutes(hours) {
  return hours * 60;
}
toMinutes(2)
toMinutes(1.5)
toMinutes(0)
```



The Console prints `undefined` after the definition. That is its comment on the line you typed, not an answer from your function. The three calls each hand back a number. (real output, rendered)

Type the definition into your browser Console, then call it with `toMinutes(3)`. Predict the answer before you press Enter.

3. What is a parameter, and what is an argument?

They are two words for the two ends of the same input.

A **parameter** is the name written in the definition: `hours`. An **argument** is the real value you give when you call: the `2` in `toMinutes(2)`. When the call runs, the parameter works like a variable that was just set to the argument. Call it again with `1.5`, and this time `hours` holds `1.5`.

© 2026 559 Solutions LLC

In the picture under Concept 1, `hours` is the parameter and `2` is the argument. Inside that one call, `hours * 60` becomes `2 * 60`.

Predict what `toMinutes()` gives with **no** argument. Then try it. (You will see `NaN`, "not a number," because `hours` holds `undefined` and `undefined * 60` is not a number.)

4. What does `return` do, and why is `console.log()` not the same?

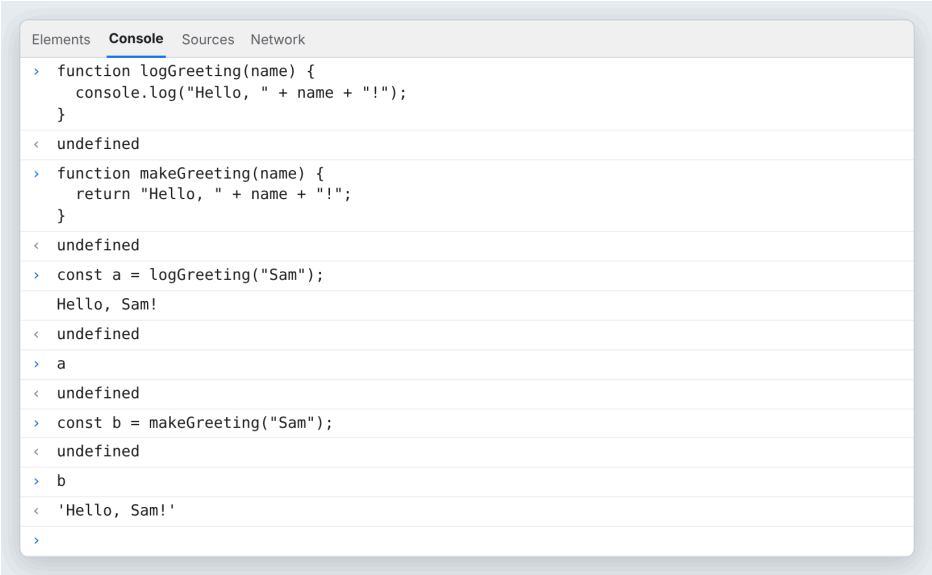
`return` hands a value back to the line that called the function, and it ends the function right there. `console.log()` only **shows** a value on the screen.

A shown value is gone as soon as you read it. A returned value can be stored in a variable, compared, joined to text, or passed to another function. If a function has no `return`, it still hands something back: `undefined`. This is the most common mistake at this stage. The function "works" because you see the right text, but the line that called it receives nothing.

Typed into the Console for the picture below, in this order:

```
function logGreeting(name) {
  console.log("Hello, " + name + "!");
}
function makeGreeting(name) {
  return "Hello, " + name + "!";
}
const a = logGreeting("Sam");
a
const b = makeGreeting("Sam");
b
```


© 2026 559 Solutions LLC



`logGreeting` shows the text and hands back `undefined`, so `a` holds `undefined`. `makeGreeting` shows nothing and hands back the text, so `b` holds `'Hello, Sam!'`. (real output, rendered)

In your Console, type `makeGreeting("Sam") + " Welcome back."` and predict the answer. Then try the same with `logGreeting`. Why does the second one end in `undefined Welcome back.`?

5. What is local scope?

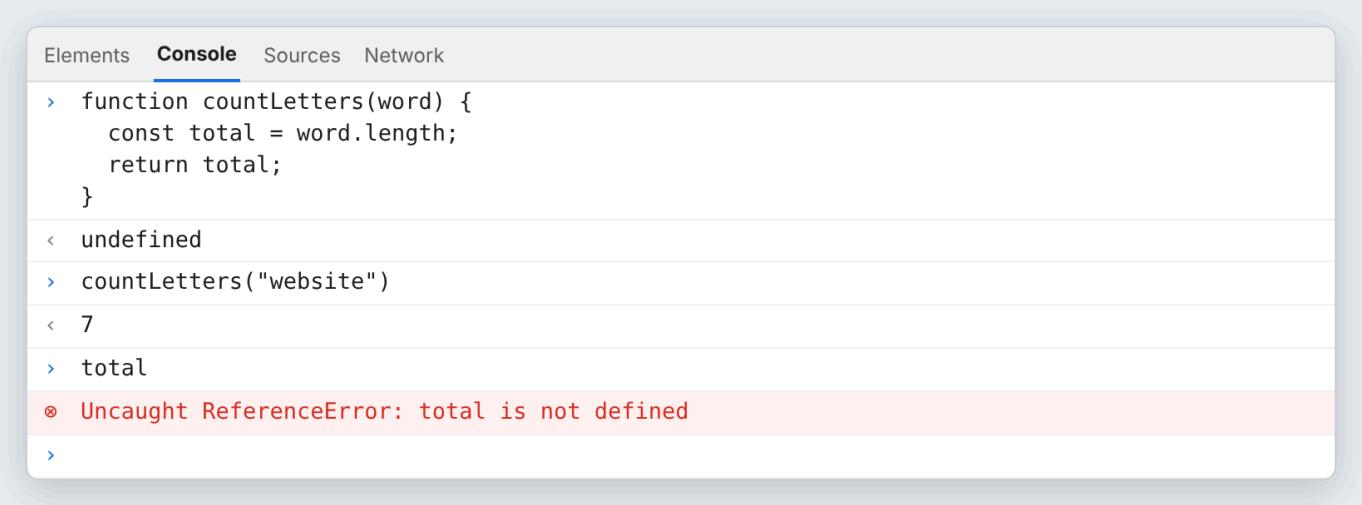
A variable created inside a function belongs to that function. Code outside cannot see it.

MDN's *Functions* guide says "variables defined inside a function cannot be accessed from anywhere outside the function." That is a feature. It means two functions can each have a variable called `total` without stepping on each other. If you need a value from inside, `return` it, and store it where you called.

Typed into the Console for the picture below, in this order (`word.length` gives the number of characters in `word`; Module 8 covers `.length` in full):

```
function countLetters(word) {
  const total = word.length;
  return total;
}
countLetters("website")
total
```


© 2026 559 Solutions LLC



The function used `total` and handed back `7`. Outside, `total` does not exist, so JavaScript stops with a `ReferenceError`. (real output, rendered)

Fix it the right way: type `const letters = countLetters("website");` and then `letters`. You get the value without reaching inside.

6. How do you test a function by itself?

You call it with several inputs, one after another, and compare each answer with what you expected.

Pick at least three inputs: a normal one, a different normal one, and an edge. An **edge** is the value right at a boundary, like `0`, an empty string, or exactly the limit in an `if`. Edges are where mistakes hide. In the Console, you type each call. In a file, you put each call inside `console.log()` and run the file with Node, so the tests can be run again any time.

Here are three tests for `countLetters` from Concept 5, written in a file. The comment holds the predictions, and the empty string `""` is the edge.

```
function countLetters(word) {
  const total = word.length;
  return total;
}

// expect 7, 2, 0
console.log(countLetters("website"));
console.log(countLetters("Hi"));
console.log(countLetters(""));
```

When you run this file with Node, it prints `7`, `2` and `0`, one per line, so all three answers match the predictions.

© 2026 559 Solutions LLC

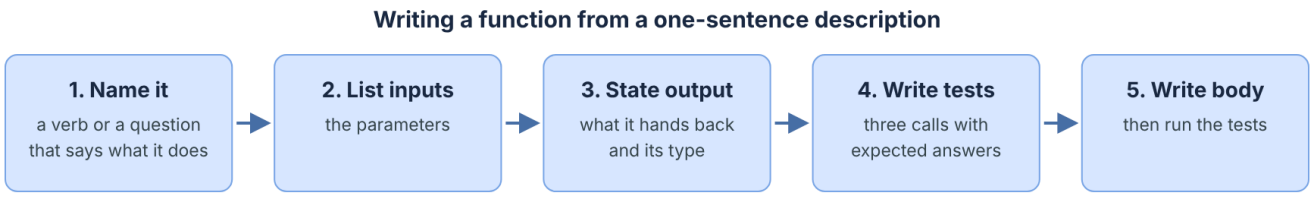
For `toMinutes`, write down three inputs and the answer you expect for each before you run anything. That list is your test.

7. How do you write a function from a one-sentence description?

This is the step most learners find hardest, because the description does not tell you where to start. Use the same five steps every time.

Take this description: *"Write a function that takes a width in pixels and returns true when it is narrower than 600."* **1. Name it** with a verb or a question: `isNarrow`. **2. List the inputs**: one, a width, so one parameter, `width`. **3. State the output and its type**: `true` or `false`, a Boolean. **4. Write the test calls first**, with the answers you expect: `isNarrow(390)` → `true`, `isNarrow(800)` → `false`, `isNarrow(600)` → `false` (the edge). **5. Write the body** last, then run the tests.

```
function isNarrow(width) {
  return width < 600;
}
```



Steps 1 to 4 need no code at all. By the time you write the body, you already know what "working" means.

Use the five steps on this description, on paper first: *"Write a function that takes a first name and returns a greeting that says Welcome, then the name, then a period (`.`)."*

8. What is the difference between `showMessage` and `showMessage()`?

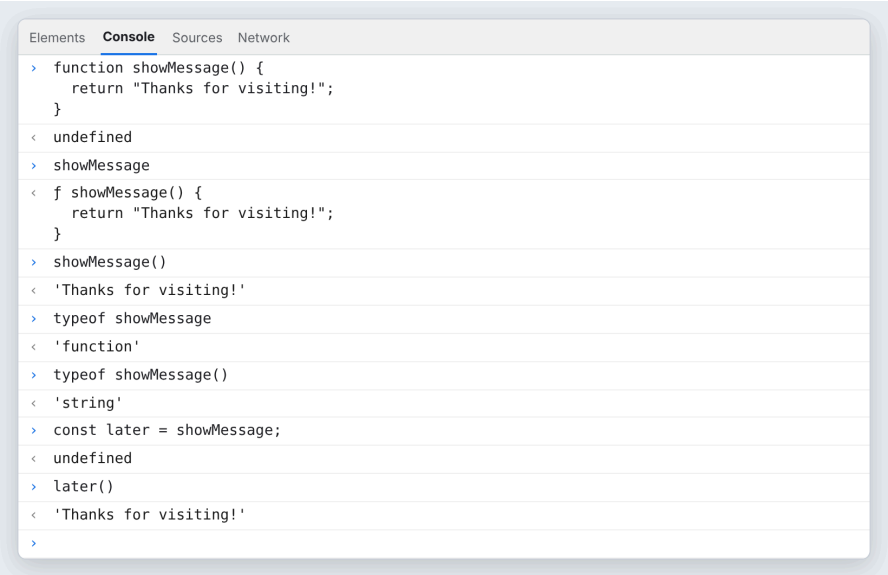
The name alone is a **reference** to the function. The name with parentheses is a **call**. They give you two completely different things.

© 2026 559 Solutions LLC

`showMessage` gives you the function itself, the stored instructions. Nothing runs. `showMessage()` runs the function and gives you whatever it returns, here a string. You can even store the function itself in another variable and call it later through the new name.

Typed into the Console for the picture below, in this order:

```
function showMessage() {
  return "Thanks for visiting!";
}
showMessage
showMessage()
typeof showMessage
typeof showMessage()
const later = showMessage;
later()
```



Without parentheses you get the function. With them you get its answer. `typeof` proves it: `'function'` versus `'string'`. In Node, `console.log(showMessage)` prints `[Function: showMessage]` instead (see the picture in Exercise step 5). (real output, rendered)

This is the bridge to Module 9. There you will tell a button: "when you are clicked, run this function." The button needs **the function itself**, so it can call it later, every time someone clicks. It does not need the result of running it once, right now.

© 2026 559 Solutions LLC

Module 9 preview: what you hand to an event listener

```
button.addEventListener("click", showMessage);
```

You hand over the function itself.
The browser keeps it and calls it
each time the button is clicked.

```
button.addEventListener("click", showMessage());
```

showMessage() runs right now, once.
You hand over its result, a string,
not something the browser can call.

A preview of Module 9. You do not need to type this yet. When we tried the right-hand version in Chromium, it refused with "TypeError: Failed to execute 'addEventListener' on 'EventTarget': parameter 2 is not of type 'Object'." If the function returns nothing, there is no error message at all: the function runs once right away, and later clicks do nothing.

Before you press Enter on each, predict what `later` and `later()` will show.

9. How does a function say "there is no answer"?

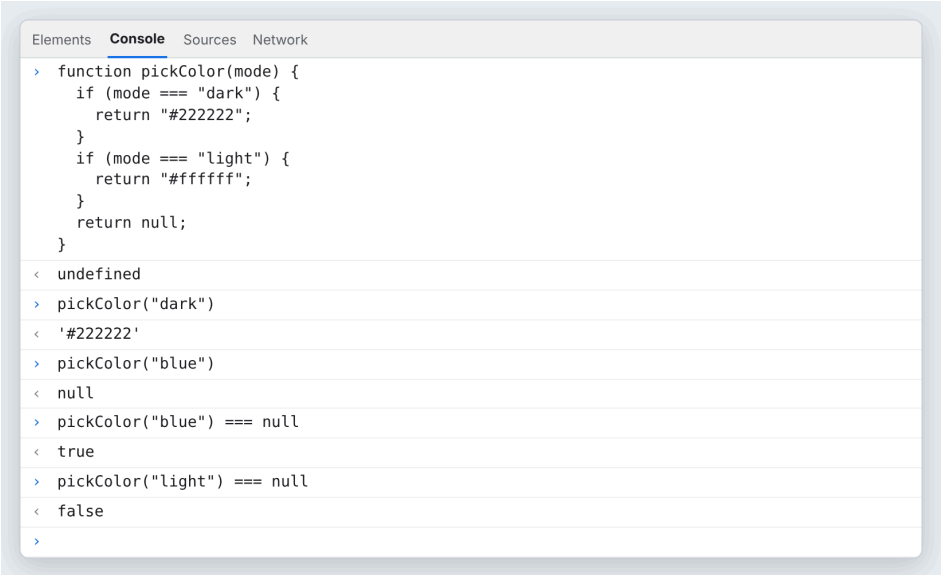
Sometimes a function is asked a question it cannot answer. It should say so clearly, with a value that means "nothing here." That value is `null`.

Both `null` and `undefined` mean "no value," but they have different jobs. `undefined` is what you get by accident, such as a forgotten `return`. `null` is a value you hand back **on purpose**. The calling code then checks with `=== null` before it uses the answer. Do not use `0` or `""` for "no answer": those can be real answers.

Typed into the Console for the picture below, in this order:

```
function pickColor(mode) {
  if (mode === "dark") {
    return "#222222";
  }
  if (mode === "light") {
    return "#ffffff";
  }
  return null;
}
pickColor("dark")
pickColor("blue")
pickColor("blue") === null
pickColor("light") === null
```


© 2026 559 Solutions LLC



`"blue"` *is not a mode this function knows, so it reaches the last line and returns* `null`*. The* `=== null` *check turns that into a plain* `true` *or* `false` *you can use in an* `if`*. (real output, rendered)*

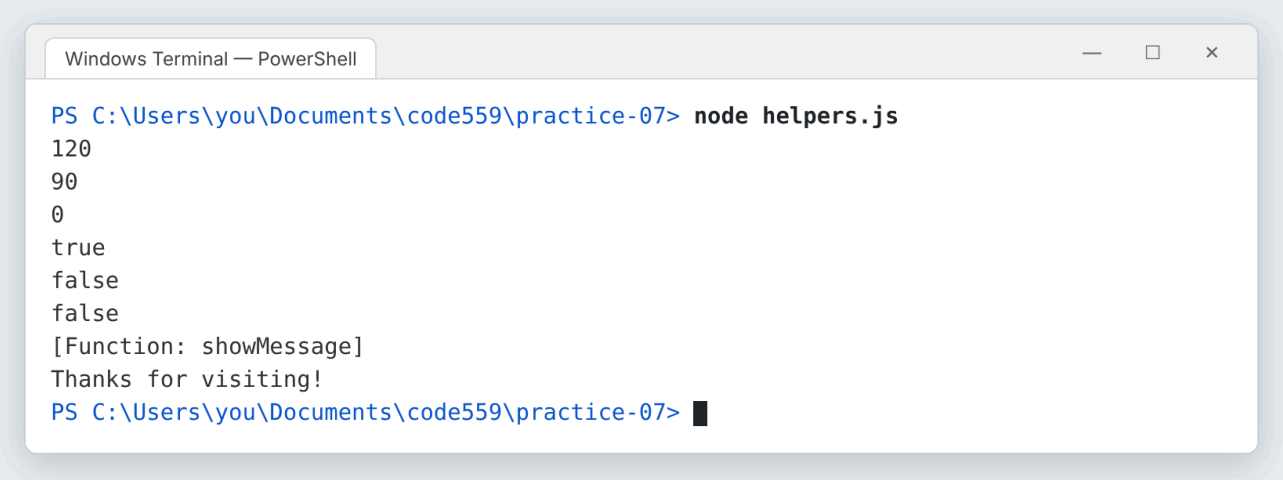
Store an answer first: `const color = pickColor("blue");`. Then write the `if` you would put in front of using the color: `if (color === null) { … }`. What should happen inside it?

Exercise

Your hands, about 45 minutes (estimate). Type it, do not paste it. Work in `practice-07`. Nothing here is published.

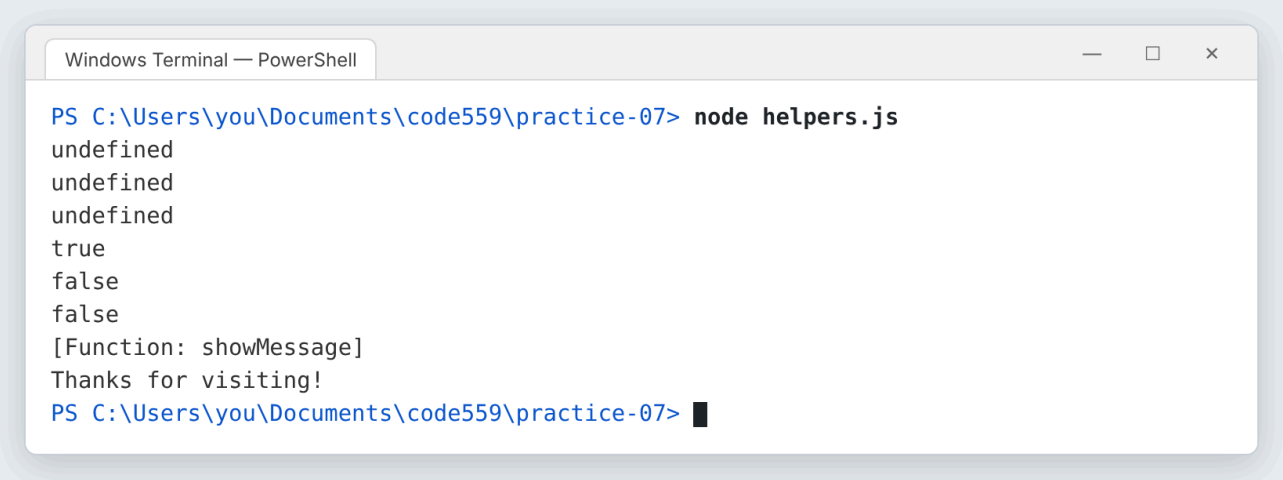
1. **Create the file.** In VS Code, create `helpers.js` inside `practice-07`.
2. **Define and test `toMinutes`.** Type the function from Concept 1. Below it, type three test lines: `console.log(toMinutes(2));`, `console.log(toMinutes(1.5));` and `console.log(toMinutes(0));`. Write your three predictions in a comment above them, like `// expect 120, 90, 0`.
3. **Run it.** Save, then in the terminal type `node helpers.js`. Compare each line with your comment.
4. **Write `isNarrow` from its description.** Use the five steps from Concept 7, in order. At the bottom of the file, write your tests first, as a comment with your predictions: `// expect isNarrow(390) true, isNarrow(800) false, isNarrow(600) false`. Under the comment, type the function. Under the function, type the three `console.log` test lines. Run the file again.
5. **Reference versus call.** Add `showMessage` from Concept 8, then two lines: `console.log(showMessage);` and `console.log(showMessage());`. Predict both lines, then run. Your output should now match this picture.

© 2026 559 Solutions LLC



One run, eight answers. The `false` for `isNarrow(600)` is the edge test: 600 is not less than 600. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

6. **Break it on purpose.** In `toMinutes`, delete the word `return` so the body reads `hours * 60;`. Save and run.



The work `hours * 60` still happens, but nothing hands the answer back, so each call gives `undefined`. No error message. That is what makes this mistake easy to miss. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Put `return` back, run once more, and check that 120, 90 and 0 return.

7. **Scope, in the Console.** In the browser Console, type `countLetters` from Concept 5, call it, then type `total`. Read the whole red message. Which word in it tells you the problem?

8. **Your own.** Write one new function from this description, with its tests first: "*Write a function that takes a number of minutes and returns true when it is an hour or more.*" Include the edge, `60`.

© 2026 559 Solutions LLC

Common misconception

> *"My function works. I can see the right answer in the Console. Adding `return` is extra work."*

What would you say? Hint: store the result in a variable and look at what the variable holds. Then think about Module 9, where the code that calls your function is not you reading the screen.

Summary

A function is a named, reusable set of instructions. Defining it stores the instructions; calling it with parentheses runs them. A parameter is the name in the definition, and an argument is the value you send in the call. `return` hands a value back and ends the function, while `console.log()` only shows a value, so a function that only logs gives `undefined` to whoever called it. Variables made inside a function stay inside it; outside, they cause a `ReferenceError`. You test a function by calling it with several inputs, including an edge, and comparing each answer with your prediction. To write a function from a description, name it, list its inputs, state its output, write the tests, and only then write the body. `showMessage` is the function itself and `showMessage()` is its result, and an event listener needs the first. A function returns `null` on purpose when there is no answer, and the caller checks with `=== null`.

Check your understanding

1. `const x = logGreeting("Ana");` prints `Hello, Ana!` but `x` holds `undefined`. Why?
2. What is the difference between what you get from `isNarrow` and from `isNarrow(390)`?
3. **What would you test** to be sure `isNarrow` is correct right at the boundary? Name the input and the answer you expect.

Before next module

- [ ] `helpers.js` in `practice-07` runs with `node helpers.js` and every line matches your predictions
- [ ] You wrote `isNarrow` and the "an hour or more" function with their tests first
- [ ] You saw `undefined` appear when `return` was removed, and put it back
- [ ] You can explain, without notes, why a listener gets `showMessage` and not `showMessage()`
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

© 2026 559 Solutions LLC

Go further

- MDN: *Functions* (the JavaScript Guide page) — read "Defining functions," "Calling functions" and "Function scopes and closures"
- MDN: *return* (the statement reference page)
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

What to keep

Keep `helpers.js` with its predictions in comments, and your paper notes from the five-step method. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "Functions" (JavaScript Guide); MDN "return"; Chromium (via Playwright) for every Console result and the addEventListener TypeError; Node 22.22.0 for every terminal result. Checked 2026-10-11: MDN "String: length"; MDN "Functions" (section titles); MDN "EventTarget: addEventListener() method" (the listener may be `null`); Chromium 141 for `addEventListener("click", sayHi())` with a function that returns nothing (no error, one run at once, clicks do nothing) and for the Concept 9 Apply; Node 22.22.0 for the `countLetters` tests in Concept 6. Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩

© 2026 559 Solutions LLC

PART 2

Project: A tested helpers.js

Your hands, about 75 minutes (estimate). In the Exercise you wrote `toMinutes` and `isNarrow` in `practice-07`, tested them in Node, and broke one on purpose. Now you write four small functions for your own site, in a file called `helpers.js`. Each one starts as a one-sentence description, the kind of task the lesson called the hardest step. You take each description through the five steps from Core concept 7, and you write its test calls **before** the function exists. Then you write the body, run the file with Node, and compare every line with your predictions. You use only what Modules 5 to 7 taught: comments, `console.log()`, `const`, joining text with `+`, `===`, `<` and `<=`, `if` and `else`, and functions with parameters, `return` and `null`.

This project goes in `my-site`, next to `about-me.js` and `plan-my-day.js`. Any `my-site` that is a Git repository connected to GitHub works, because no page uses the file yet.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds a new file, `helpers.js`, at the top level, next to `index.html`. It is a new file. It has the same name as the Exercise file in `practice-07`, but it is not a copy of it. No page links to it and no page runs it yet: like your other scripts, it runs in the terminal only.

The four descriptions. Each helper is one sentence. Use the names given here, so your tests and the answers in Part 6 line up.

1. **`toFileName`.** Write a function that takes a page name in lowercase and returns its file name: the name followed by `.html`, except `home`, which returns `index.html`.
2. **`makePageTitle`.** Write a function that takes a page name and your name and returns the text for the browser tab: the page name, a space, a dash, a space, then your name. This is the `Home - Your Name` pattern your pages have used since Module 1.
3. **`isNarrowScreen`.** Write a function that takes a screen width in pixels and returns `true` when it is narrower than 700, the width where your site's wider layout starts (the `@media (min-width: 700px)` block from Module 3).
4. **`navLabel`.** Write a function that takes a file name and returns the words on its link in your nav, or `null` when your site has no page with that file name.

What it prints. Run from the `my-site` folder, `node helpers.js` printed exactly these 21 lines for the sample student, Sam:

```
--- toFileName ---
index.html
about.html
contact.html
```


© 2026 559 Solutions LLC

```
--- makePageTitle ---
Home - Sam
About - Sam
Rate this lesson - Sam
--- isNarrowScreen ---
true
false
false
true
--- navLabel ---
Home
Contact
null
null
true
false
blog.html is not a page on this site
```

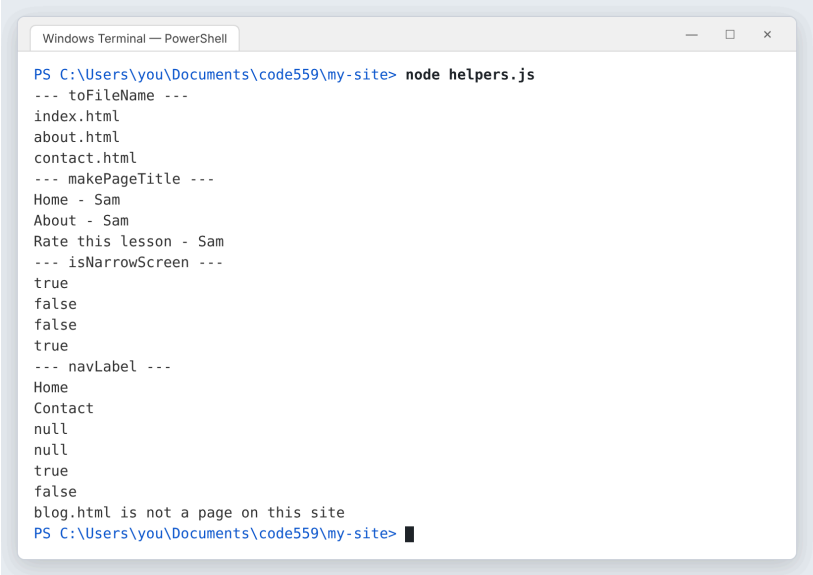
Your file prints the same shape with your own name and your own pages:

- **One group of test lines per helper**, in the order of the descriptions. Each group starts with a heading line, such as `--- toFileName ---`, so you can tell the groups apart.
- **At least three tests per helper, and one of them an edge.** For `toFileName` the edge is `home`, the one name that breaks the pattern. For `isNarrowScreen` it is 700 and 699, one on each side of the limit. For `navLabel` it is a file name your site does not have.
- **Two `=== null` checks** in the `navLabel` group: one that gives `true` and one that gives `false`.
- **A last line that uses two helpers together**, and checks for `null` before it uses an answer.

What is inside the file.

- The first two lines are comments: the file name with one sentence about what it does, and how to run it.
- Three parts, each under its own heading comment, in this order: `// ===== The helpers =====`, `// ===== The tests =====` and `// ===== Using two helpers together =====`.
- Above each function, a comment with its name and its description, in your words.
- Above each group of tests, a comment that starts with `// expect` and lists your predictions, in order.
- Every function hands its answer back with `return`. No function has a `console.log()` inside it: only the test lines print.
- Only `const`, never `let` and never `var`. Comparisons use `===`, never `==`.

© 2026 559 Solutions LLC



The finished file for the sample student, in one run. Each line matches its `// expect` comment. The `null` lines are answers too: `navLabel` handed back "no answer" on purpose. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

The steps

Type every line. Do not paste. You may copy a test line and change only its argument, once you have typed the first test line of a group yourself.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.

2. **Plan on paper.** For each of the four descriptions, do steps 1 to 3 of the five-step method from Core concept 7. Make a small table with four rows and three columns: **Name**, **Inputs** (one parameter name for each input) and **Output and its type**. The names are given in the brief, so step 1 is done. For the inputs, choose names that say what the value is, such as `pageName` or `width`. For the output, write the type as well as an example, such as "a string, like `about.html`". Do not write any code yet. This page is your plan.

3. **Start the file.** In the VS Code Explorer, create `helpers.js` at the top level of `my-site`, next to `index.html`. Type these lines, then save with Ctrl+S:

```
// helpers.js: four small helper functions for my site, with their tests
// Run it from the my-site folder with: node helpers.js

// ===== The helpers =====

// ===== The tests =====
```


© 2026 559 Solutions LLC

Functions go under the first heading and test lines under the second, so the tests always run after every function is defined. **Predict** before you run: what will `node helpers.js` print? Write it down. Then type `node helpers.js` and press Enter. The prompt came straight back. Only comments, so nothing to run.

4. **`toFileName` : the tests first, then the body.** This is step 4 of the five-step method. Under `// ===== The tests =====`, leave one blank line and type:

```
console.log("--- toFileName ---");
// expect index.html, about.html, contact.html
console.log(toFileName("home"));
console.log(toFileName("about"));
console.log(toFileName("contact"));
```

The `// expect` comment holds your predictions, in the same order as the test lines. Each test line calls the function inside `console.log()`, so its answer prints. **Think this through:** the function does not exist yet. What will Node do? Write down your answer. Save and run. The sample printed the heading line, and then this (the `at` lines below it are left out):

```
C:\Users\you\Documents\code559\my-site\helpers.js:10
console.log(toFileName("home"));
        ^

ReferenceError: toFileName is not defined
```

This is the Module 5 error: JavaScript does not know the name yet. The caret sits under `log`, not under the missing name, so read the message: it names `toFileName`. Your line number may differ. A test that fails before the function exists is a good start. You now know exactly what "working" means: three lines that match your comment.

Now step 5: the body. Under `// ===== The helpers =====`, leave one blank line and type the function with its comment:

```
// toFileName: takes a page name in lowercase and returns its file name,
// the name followed by .html, except home, which is index.html.
function toFileName(pageName) {
  if (pageName === "home") {
    return "index.html";
  }
  return pageName + ".html";
}
```


© 2026 559 Solutions LLC

Leave one blank line after the closing `}`. Read the body and say what happens for `"home"`: the condition is true, so `return "index.html"` hands back the answer and ends the function right there (Core concept 4). The last line never runs for `"home"`. For any other name, the condition is false, so the last line joins the name and `".html"` and returns that. Save and run. Compare each line with your comment: the sample printed `--- toFileName ---`, `index.html`, `about.html` and `contact.html`.

5. **`makePageTitle`: two parameters.** Under the `toFileName` tests, leave one blank line and type the tests first, with your own name in place of Sam's:

```
console.log("--- makePageTitle ---");
// expect Home - Sam, About - Sam, Rate this lesson - Sam
console.log(makePageTitle("Home", "Sam"));
console.log(makePageTitle("About", "Sam"));
console.log(makePageTitle("Rate this lesson", "Sam"));
```

The last test is the long one: a page name with spaces in it, like the title of the Module 4 rate page. Save and run once before you write the function. The whole `toFileName` group prints, then the new heading line, then `ReferenceError: makePageTitle is not defined`. The tests above the bad line ran and passed. Node stopped only where it met the name it did not know.

Then, under the `toFileName` function, leave one blank line and start the function:

```
// makePageTitle: takes a page name and your name and returns the text
// for the browser tab: the page name, a dash, then your name.
function makePageTitle(pageName, ownerName) {
  // Continue alone: one line that returns the page name,
  // the text " - " and the owner's name, joined with +.
}
```

Two parameters, separated by a comma. In a call, the arguments are matched to the parameters **by position**: the first argument goes into `pageName` and the second into `ownerName` (Core concept 3). Write the body alone. Mind the spaces inside `" - "`: without them, the output reads `Home-Sam`. Delete the two "Continue alone" comment lines, save and run. The sample printed `Home - Sam`, `About - Sam` and `Rate this lesson - Sam` under the heading.

6. **`isNarrowScreen`: create it alone.** This one has no code to copy, only the description from the brief. Do all five steps yourself:

   - In the tests part, under the `makePageTitle` tests, type a heading line `--- isNarrowScreen ---`, an `// expect` comment and at least four test lines. Use a phone width, a laptop width, and the two edge widths: exactly 700 and 699. Fill in the `// expect` comment before you run anything.
   - Run once and see the `ReferenceError` for the new name, as before.

© 2026 559 Solutions LLC

- In the helpers part, under `makePageTitle`, type the comment and the function. The answer is a Boolean, so the body can be a single `return` with a comparison, like `isNarrow` in the lesson.

Save and run. The sample tested 390, 1200, 700 and 699, and printed `true`, `false`, `false` and `true`. The `false` for 700 is the edge test: 700 is not narrower than 700, and at exactly 700 your wider layout already applies, because `min-width` includes the number itself. If your 700 line prints `true`, compare your comparison sign with the description.

7. **`navLabel` : a function that can say "no answer."** In the tests part, under the `isNarrowScreen` tests, type the tests first:

```
console.log("--- navLabel ---");
// expect Home, Contact, null, null, true, false
console.log(navLabel("index.html"));
console.log(navLabel("contact.html"));
console.log(navLabel("blog.html"));
console.log(navLabel("About.html"));
console.log(navLabel("blog.html") === null);
console.log(navLabel("about.html") === null);
```

Use the file names of your own site. The sample site has five pages from the Modules 1 to 4 packets: `index.html`, `about.html`, `interests.html`, `rate.html` and `contact.html`. If your site has no `contact.html`, test one of your own pages instead and change the `// expect` comment to match. `blog.html` is the page the site does not have. `About.html`, with a capital A, is the edge: `===` compares letter for letter, so it is not the same as `about.html`. The last two lines turn the answer into a plain `true` or `false`, as in Core concept 9. Run once and see the `ReferenceError`. Then, in the helpers part, under `isNarrowScreen`, start the function:

```
// navLabel: takes a file name and returns the words for its nav link,
// or null when the site has no page with that file name.
function navLabel(fileName) {
  if (fileName === "index.html") {
    return "Home";
  }
  // Continue alone: one if for each other page your site has,
  // in the same order as your nav.
  return null;
}
```

Each `if` returns at once when its file name matches, and the function ends there. Only a file name that matches none of them reaches the last line, `return null;`. That is the "no answer" on purpose from Core concept 9. Write one `if` block for each of your other pages, with the nav

© 2026 559 Solutions LLC

words exactly as they are on your links. Delete the two "Continue alone" lines, save and run. The sample printed `Home`, `Contact`, `null`, `null`, `true` and `false` under the heading.

This function has more braces than any you have written. If Node prints a `SyntaxError`, nothing at all prints, not even the first heading line, because Node could not read the file. Read the message, the line it names, and the lines just above it. The two most likely messages here:

- `SyntaxError: Unexpected token '}'`: there is a `}` too many, or a `{` is missing after an `if (...)` line. Either way, a `}` closes the function too early, and Node complains at a later `}`. The line it names can be several lines below the real mistake, often the function's last line. Check every `if` line ends in `{`, and count: every `{` needs its own `}`.
- `SyntaxError: Unexpected end of input`, pointing at the end of the file (the empty line after the last `}`): a `}` is missing, or there is an extra `{`. Node reached the end of the file with a block still open.

For any other `SyntaxError` message, see Problem 6 in Part 4, *Troubleshooting*.

8. **Use two helpers together, and check for `null` first.** Under the last test line, leave one blank line and type:

```
// ===== Using two helpers together =====

console.log(makePageTitle(navLabel("blog.html"), "Sam"));
```

The answer of `navLabel` goes straight into `makePageTitle` as its first argument. **Think this through:** what does `navLabel("blog.html")` give, and what will `makePageTitle` do with it? Write down your answer, then save and run. The sample's last line was `null - Sam`. No error, but a wrong answer: `+` joined `null` into the text as the word `null`. A visitor would see a tab called `null - Sam`. This is why the lesson says to check with `=== null` before you use the answer. Replace the line with this:

```
const label = navLabel("blog.html");
if (label === null) {
  console.log("blog.html is not a page on this site");
} else {
  console.log(makePageTitle(label, "Sam"));
}
```

Save and run. The last line is now `blog.html is not a page on this site`. Then change `"blog.html"` to `"about.html"` on the `const label` line, and only there. Predict the last line, save and run. The sample printed `About - Sam`: the answer was not `null`, so the `else` block used it. The message in the `if` block still says `blog.html`, so change the file name back to `"blog.html"`, save and run once more.

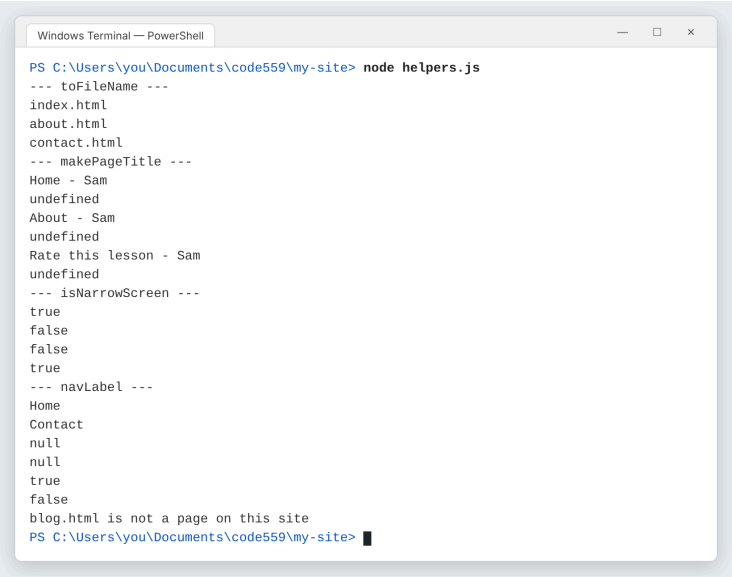
© 2026 559 Solutions LLC

9. **Modify.** Make two changes, one at a time. Predict each before you run it, then undo it.

    - **Change `<` to `<=`.** In `isNarrowScreen`, change `width < 700` to `width <= 700`. Save. Which test line would show this change? Write it down, then run. In the sample, only the third `isNarrowScreen` line changed, from `false` to `true`: the test for 700. The tests for 390, 1200 and 699 gave the same answers as before. This is why the edge test is there. Change it back, save and run.
    - **Swap two arguments.** In the second `makePageTitle` test, swap the two arguments, so the line reads `console.log(makePageTitle("Sam", "About"));`. Predict, save and run. The sample printed `Sam - About`. JavaScript does not read the parameter names to decide where a value goes. It goes by position: the first argument always becomes `pageName`. Swap them back, save and run.

10. **Break it on purpose.** Two classic mistakes from the lesson. Make each one, run, look, and undo it before the next.

    - **`console.log()` instead of `return`.** In `makePageTitle`, change the word `return` to `console.log(`, and add a `)` before the semicolon, so the body is `console.log(pageName + " - " + ownerName);`. **Think this through:** what will the `makePageTitle` group print now? Write it down, then save and run.



    Six lines where there were three, and no error message. Each title prints from inside the function, and then the test line prints what the function handed back: `undefined`. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

    Each call now does two things. The `console.log()` inside the function shows the title. Then the function ends with no `return`, so it hands back `undefined`, and the test line prints that. The titles on the screen look right, but the code that called the function received nothing. This is the misconception at the end of the lesson. Put `return` back, save and run, and check that the group shows three titles and no `undefined`.

© 2026 559 Solutions LLC

- **No parentheses.** In the `toFileName` tests, delete the parentheses and the argument from the second test, so it reads `console.log(toFileName);`. Predict, save and run. The sample printed `[Function: toFileName]` where `about.html` had been. Without parentheses, the name is a reference to the function itself, so Node printed the function, not an answer (Core concept 8). Nothing ran. Put back `("about")`, save and run.

11. **Debug: a test that fails with no error.** Here is a mistake that is easy to make in `navLabel`. Change the first `if` block so it has an `else`:

    ```
    if (fileName === "index.html") {
      return "Home";
    } else {
      return null;
    }
    ```

    Leave the other `if` blocks and the last `return null;` as they are. Save. Before you run, write down which lines of the `navLabel` group you expect to change. Then run. The sample printed `Home`, `null`, `null`, `null`, `true` and `true`. Compare with the `// expect` comment, one line at a time: the second line should be `Contact`, and the last line should be `false`. Two tests failed, and Node showed no error.

    When a test fails, first ask which one is wrong: your prediction or your code. Here the prediction is right, because `contact.html` is a page on the site. So the code is wrong. Follow the call `navLabel("contact.html")` line by line, as JavaScript does. `"contact.html" === "index.html"` is false, so the `else` block runs: `return null`. `return` ends the function right there, so the `if` blocks for the other pages are never reached. With this `else`, every file name except `index.html` gets `null`. Delete the `else` part, from `} else {` to its closing `}`, and put back the single `}` that closes the first `if`. Save, run, and check all six lines against your comment. In your notebook, write one sentence on why a test with only `index.html` could not catch this bug.

12. **Explain it.** With every test passing, read the whole file to yourself, out loud, from top to bottom. For each function, say its parameters, what type of value it returns, and which test line checks its edge. For each `return`, say when it runs and what it ends. For `navLabel`, say why `return null;` must be the last line and not the first. If there is a line you cannot explain, reread the Core concept it comes from.

13. **Looking ahead to Module 9.** No page uses `helpers.js` yet. In Module 9 you learn how a page loads a script file with a `<script>` tag, and how a click runs a function. The functions there are built from the same parts as these: a name, a body, and sometimes parameters. Two habits from this project carry over. First, test a function by itself, in Node, where a wrong answer is easy to see, before any page depends on it. Second, when a click should run a function, hand over the

© 2026 559 Solutions LLC

function itself, not its answer: `showMessage`, not `showMessage()`, as Core concept 8 showed. The test lines are for you, not for visitors. You run them with Node, after every change, to prove the helpers still work.

14. **Check the file.** Tick each box.

    - [ ] `helpers.js` is in `my-site`, next to `index.html`, not in `practice-07`
    - [ ] The prompt ends in `my-site>`, and `node helpers.js` prints four groups and the last line, with no error
    - [ ] Every test line matches its `// expect` comment, in order
    - [ ] Each helper has at least three tests, one of them an edge: `home`, 700 and 699, and a file your site does not have
    - [ ] `navLabel` returns `null` for a page your site does not have, and the file checks it with `=== null` before using it
    - [ ] Every function hands back its answer with `return`; there is no `console.log()` inside any function
    - [ ] Each function has a comment above it with its description
    - [ ] The `else` from step 11 is gone, and the parentheses from step 10 are back
    - [ ] Nothing in the file is private: your first name or a nickname is enough

15. **Save it in Git.** Go to Part 3, *Finishing work in Git*, and save the file with the save routine.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Every test in `helpers.js` passes. Now you save it to GitHub with the same save routine as in Modules 1 to 6: `git status`, `git add .`, `git commit`, `git push`, then check the live site. What is new this time: the tests are part of the file you save, so you run them as the last thing before every commit. And you add one more test after the first commit, then save again, to see how a small change looks to Git.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 6.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **Run the tests one last time (NEW: the tests decide when you commit).** Type `node helpers.js` and read every line against its `// expect` comment. Look for the three signs of the mistakes from Project steps 10 and 11: an `undefined` where an answer should be, a `[Function: …]` line, or a `null` your comment did not predict. Check one more thing in the file: the `const label` line uses `"blog.html"` again, as in Project step 8. Commit only a file whose tests you have just run and seen pass. The test lines stay in the file: they are your proof, and you can run them again after any change.

3. **See what changed.** Type:

   ```
   git status
   ```

   Under **Untracked files** you should see one line: `helpers.js`. Git has never saved it before.

   NEW: two files with the same name. You now have two files called `helpers.js`: the Exercise file in `practice-07`, and the project file in `my-site`. Git sees only the one inside `my-site`, because the repository is `my-site`. The `practice-07` folder never shows up here. Before you go on, check that the right file is open: in the VS Code Explorer, `helpers.js` should sit at the top level of `my-site`, next to `index.html`.

4. **Stage the change.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** you should now see `new file: helpers.js`.

5. **Commit.** Type:

   ```
   git commit -m "Add helpers.js with four helper functions, each tested in Node"
   ```


© 2026 559 Solutions LLC

On the sample site, Git printed `1 file changed, 85 insertions(+)` on the second line and `create mode 100644 helpers.js` on the third. The 85 is the number of lines in the sample file, comments, tests and blank lines included. Your commit id and line count will differ.

6. **Push.** Type:

   ```
   git push
   ```

   The last line ends in `main -> main`.

7. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 6 commit (or your latest one) under it.

8. **Add one more test, and save again (NEW: a small second commit).** Tests grow as a site grows. Add a fourth test to the `toFileName` group, for one of your other pages, and add its prediction to the `// expect` comment. The sample's group became:

   ```
   // expect index.html, about.html, contact.html, interests.html
   console.log(toFileName("home"));
   console.log(toFileName("about"));
   console.log(toFileName("contact"));
   console.log(toFileName("interests"));
   ```

   Save, run `node helpers.js`, and check that the new line matches the comment. Then type `git status`. This time it lists `modified: helpers.js` under **Changes not staged for commit**. Run the routine again, with a message that names the change:

   ```
   git add .
   git commit -m "Add a toFileName test for the Interests page"
   git push
   ```

   On the sample, the commit printed `1 file changed, 2 insertions(+), 1 deletion(-)`. Git counts a changed line as one line deleted and one line added: the `// expect` comment changed, and one test line is new. `git log --oneline` now shows both commits on top, the newest first. One small change, tested, then saved with a message that says what it was: that is a commit you can find again later.

9. **Check the live site.** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/` and press F5. Your pages look exactly as before, because no page loads `helpers.js`. Then open `https://your-username.github.io/my-site/helpers.js`. As with your scripts in Modules 5

© 2026 559 Solutions LLC

and 6, the browser shows your code as text or offers to download it, and nothing runs. Anyone with the address can read it, your tests included, so check once more that it holds nothing private.

Checkpoint. `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your two new commits on top. `node helpers.js` still prints every group with no error, your live pages look the same as before, and the live address of `helpers.js` shows your code without running it.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: six common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose with the sample `helpers.js` and run with Node 22. The messages were captured on Linux, with the folder path changed to the course folder; on Windows, Node prints the same words with your own path. The `at` lines under each message are left out here. After each fix, save, and run `node helpers.js` again from the `my-site` folder, and check every test line against its `// expect` comment. Three of the six problems print no error at all. Your tests are what catch them.

1. undefined where an answer should be

Symptom. No error, but a test line prints `undefined`. With the word `return` deleted from `makePageTitle`, so the body was only `pageName + " - " + ownerName;`, the sample printed:

```
--- makePageTitle ---
undefined
undefined
undefined
```

With `console.log(` in place of `return`, as in Project step 10, each title printed and was followed by an `undefined` line.

Why. A function with no `return` still hands something back: `undefined` (Core concept 4). The work in the body happens, the text is joined, and then it is thrown away. A `console.log()` inside the function shows the text, but showing is not handing back, so the test line still receives `undefined`.

Fix. Put `return` at the start of the line that makes the answer: `return pageName + " - " + ownerName;`. Then remove any `console.log()` from inside the function. In this file, only the test lines print.

2. ReferenceError for a parameter used outside its function

Symptom. Some test groups print, then the file stops. With the line `console.log(pageName);` added under the `toFileName` tests, the sample printed the `toFileName` group and then:

```
C:\Users\you\Documents\code559\my-site\helpers.js:55
console.log(pageName);
            ^

ReferenceError: pageName is not defined
```


© 2026 559 Solutions LLC

Why. A parameter belongs to its function, like a `const` made inside the body (Core concept 5). `pageName` exists only while `toFileName` or `makePageTitle` is running, and only inside it. The test lines are outside every function, so for them the name does not exist. Each call fills the parameter with a new argument, so there is no single `pageName` to look at anyway.

Fix. Delete the line. To see a value from inside a function, get it the way the lesson does: call the function and use what it returns, such as `console.log(toFileName("about"));`. If you need the answer twice, store it first: `const fileName = toFileName("about");`.

3. A missing argument: NaN, undefined in the text, or a quiet false

Symptom. No error, but a strange answer. With the second argument left out, `makePageTitle("About")` printed `About - undefined`. In the Exercise file in `practice-07`, `toMinutes()` with no argument printed `NaN`, which means "not a number." And with the argument left out of one `isNarrowScreen` test, `isNarrowScreen()` printed `false`, the same answer the sample expected from `isNarrowScreen(1200)`, so the test seemed to pass.

Why. A parameter with no argument holds `undefined` (Core concept 3). Then the body does its normal work with `undefined`. `+` joins it into text as the word `undefined`. `*` cannot do math with it, so the answer is `NaN`. And `undefined < 700` is `false`, which looks like a real answer. That last one is the hardest to spot.

Fix. Compare the call with the definition: one argument for each parameter, in the same order, separated by commas. Read every test line, not only its result: a test that passes for the wrong reason proves nothing. When an answer contains the word `undefined` or `NaN`, look first for a missing argument.

4. [Function: toFileName] instead of an answer

Symptom. No error, but one test line prints the function's name in square brackets. With `console.log(toFileName);` as a test line, the sample printed `[Function: toFileName]` where `about.html` should have been.

Why. The name without parentheses is a reference to the function itself, the stored instructions (Core concept 8). Nothing runs. `console.log()` shows you what it was given: a function, named `toFileName`. The browser Console shows a function differently, as the lesson's picture does, but the cause is the same.

Fix. Add the parentheses and the argument: `console.log(toFileName("about"));`. Every call needs `()`, even a function with no parameters, such as `showMessage()`. Keep the habit from the other side too: in Module 9, an event listener gets the name with **no** parentheses, because there you hand over the function itself.

© 2026 559 Solutions LLC

5. TypeError: x is not a function

Symptom. Some test groups print, then the file stops. With these two lines added under the `toFileName` tests, the sample printed the `toFileName` group and then:

```
const fileName = toFileName("about");
console.log(fileName());
```

```
C:\Users\you\Documents\code559\my-site\helpers.js:56
console.log(fileName());
            ^

TypeError: fileName is not a function
```

Why. Parentheses mean "run this function." `fileName` is not a function: it holds the string `"about.html"`, the answer that `toFileName` returned. A string cannot run, so Node stops with a `TypeError`, the error for a value of the wrong type. You get the same kind of message whenever `()` goes after something that is not a function. With an extra `()` after a call, as in `console.log(toFileName("about")());`, the sample printed `TypeError: toFileName(...) is not a function`: the `(...)` stands for the call, whose answer was a string.

Fix. Look at the name in the message, then find where that name got its value. If it holds an answer, use it without parentheses: `console.log(fileName);`. If you meant to call a function, check the spelling against its `function` line, capital letters included.

6. SyntaxError messages

Symptom. Nothing prints at all, not even the first heading line. Node shows a line of your file, a caret or carets, and a message that starts with `SyntaxError`.

Why. Node could not read the file as JavaScript, so it ran none of it. Each message below was made on purpose in the sample file.

© 2026 559 Solutions LLC

Message	Usual cause	Where to look
`Unexpected token '}'`	A `}` too many, or a `{` missing after `if (...)`	Lines above the one named, often several; the named line is often the function's last `}`
`Unexpected end of input`	A `}` missing, or an extra `{`	Every block; Node points at the end of the file
`Unexpected token 'if'`	The `{` missing at the end of a `function` line	The `function` line just above
`Unexpected token '{'`	The `)` missing before `{`, on a `function` or `if` line	The named line, just before the caret
`Unexpected token ';'`	A semicolon between `)` and `{` on a `function` line	The named line
`Invalid or unexpected token`	A missing quote, so a string never ends	The string under the carets
`Unexpected string`	A misspelled `return`, such as `retrun`	The word before the carets
`Unexpected identifier`	A missing `+` between two parts, a missing comma between two parameters, or missing parentheses around an `if` condition	The named line, just before the carets
`missing ) after argument list`	One `)` missing from `))` on a test line, or a missing comma between two arguments	The named test line

Fix. Fix the one thing the row points to, save, and run. If the message changes, read the new one from the top: one missing brace can hide another mistake below it.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 8. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can name the parts of a function definition, and explain why defining a function runs nothing until it is called	☐	☐	☐	☐
I can tell a parameter from an argument, match arguments to parameters by position, and predict what a missing argument gives	☐	☐	☐	☐
I can use return to hand back a value, and spot a function that only logs by the undefined it hands back	☐	☐	☐	☐
I can explain local scope, and fix a ReferenceError for a name made inside a function by returning the value and storing it	☐	☐	☐	☐
I can test a function with several inputs, an edge included, writing the expected answers before I run anything	☐	☐	☐	☐
I can write a function from a one-sentence description with the five steps: name, inputs, output, tests, body	☐	☐	☐	☐
I can tell a function reference from a call, read [Function: name] and "is not a function", and say why a listener gets the reference	☐	☐	☐	☐
I can return null on purpose when there is no answer, and check it with === null before using the answer	☐	☐	☐	☐
I can read a failing test, decide whether my prediction or my code is wrong, and follow the call line by line to find the bug	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 10, the `console.log()` version of `makePageTitle` printed every title and also an `undefined` line after each one. Explain where each of the two lines came from, and why the screen looking right did not mean the function was right.

© 2026 559 Solutions LLC

2. Pick one test from your project, or one prediction from the Exercise, where your first answer was wrong. What did you write down, what did Node or the Console show, and what do you understand now that you did not before?

3. A friend wrote this from the description "*Write a function that takes a number of minutes and returns the text for a reading time, like 5 min read.*" What does this file print when it runs, and what two changes fix it?

```
function readingTime(minutes) {
  const text = minutes + " min read";
}
console.log(readingTime(5));
console.log(text);
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **Why `x` holds `undefined`.** `logGreeting` has no `return`. Its body shows `Hello, Ana!` with `console.log()`, and showing is all it does. When the body ends, the function hands back the value it always hands back without a `return`: `undefined`. So `x` stores `undefined`. The text on the screen came from inside the function, not from the value it gave back. `makeGreeting("Ana")` would show nothing and give `x` the text `"Hello, Ana!"`. (Core concept 4)
2. **`isNarrow` and `isNarrow(390)`.** `isNarrow`, with no parentheses, is a reference to the function itself: the stored instructions. Nothing runs. `typeof isNarrow` gives `"function"`, and in Node, `console.log(isNarrow)` prints `[Function: isNarrow]`. `isNarrow(390)` is a call: the body runs with `width` set to 390, and you get its answer, `true`. (Core concept 8)
3. **Testing the boundary.** Test `isNarrow(600)` and expect `false`, because 600 is not less than 600. Then test the number just below it, `isNarrow(599)`, and expect `true`. The pair checks the comparison sign: with `<=` by mistake, 600 would give `true`, and every test away from the edge would still pass. (Core concepts 6 and 7)

Common misconception

"My function works. I can see the right answer in the Console. Adding `return` is extra work." Seeing the answer is not the same as getting it back. Store the result in a variable, as in Core concept 4: `const a = logGreeting("Sam");` shows `Hello, Sam!`, but `a` holds `undefined`. Any code that wants to use the answer, to join it to more text, compare it, or pass it to another function, gets nothing. Project step 10 shows the same thing in a file: with `console.log()` in place of `return`, the titles printed, and every test line printed `undefined`. In Module 9 the code that calls your functions is not you reading the screen. It is your page's script, and it can only use what the function hands back. (Core concept 4; Project step 10)

Exercise checks

- **Steps 2 and 3.** `120`, `90` and `0`, one per line, matching `// expect 120, 90, 0`. `toMinutes(1.5)` is `1.5 * 60`, which is 90. (Core concepts 2 and 6)
- **Step 4.** Name: `isNarrow`. Input: one width, so one parameter, `width`. Output: `true` or `false`, a Boolean. The three new lines are `true`, `false` and `false`. The last one is the edge: 600 is not less than 600. (Core concept 7)

© 2026 559 Solutions LLC

- **Step 5.** The two new lines are `[Function: showMessage]` and `Thanks for visiting!`. The whole run prints eight lines: `120`, `90`, `0`, `true`, `false`, `false`, `[Function: showMessage]` and `Thanks for visiting!`. The first of the two new lines is the function itself, because there are no parentheses. The second is its answer. (Core concept 8)
- **Step 6.** The first three lines become `undefined`, and the other five lines do not change. There is no error. `hours * 60` is still worked out, but with no `return`, the answer is thrown away and each call hands back `undefined`. (Core concept 4)
- **Step 7.** The Console shows `Uncaught ReferenceError: total is not defined`. The words that tell you the problem are `not defined`, with the name `total` just before them: outside the function, the name does not exist. (Core concept 5)
- **Step 8.** One possible answer, with its tests first:

  ```
  // expect isHourOrMore(90) true, isHourOrMore(30) false, isHourOrMore(60) true
  function isHourOrMore(minutes) {
    return minutes >= 60;
  }
  console.log(isHourOrMore(90));
  console.log(isHourOrMore(30));
  console.log(isHourOrMore(60));
  ```

  In Node it printed `true`, `false` and `true`. The edge is 60: "an hour or more" includes exactly one hour, so the comparison is `>=`. With `>`, the test for 60 prints `false` and fails, and the other two still pass. Any name that asks a question works, such as `isAtLeastAnHour`. (Core concepts 6 and 7)

Self-assessment questions

1. **Two lines for each title.** The first line came from inside the function: `console.log()` showed the joined text while the function ran. The second came from the test line: the function then ended with no `return`, so it handed back `undefined`, and `console.log(makePageTitle(...))` printed that. The screen showed the right titles, but no code received them. If a page used `makePageTitle` to set a tab's text, it would get `undefined`. A function is right only when its answer comes back, and the test lines check exactly that. (Core concept 4; Project step 10)
2. **A prediction you got wrong.** There is no single answer. A good answer quotes what you wrote down before the run, quotes what Node or the Console really showed, and names the idea you understand now, for example that the function returns `undefined` with no `return`, or that `return` ends a function, so an `else` with `return null` hides every `if` after it.
3. **The friend's reading time.** The file prints `undefined`, then stops with `ReferenceError: text is not defined` on the last line. There are two mistakes. First, `readingTime` has no `return`, so the joined text is thrown away and the call gives `undefined`. Second, `text` was made inside the function, so it does not exist outside it. The two changes: add `return text;` as the last line of

© 2026 559 Solutions LLC

the body, and replace `console.log(text);` with a line that stores the answer and prints it, such as `const label = readingTime(5);` and then `console.log(label);`. Then the file prints `5 min read` twice. (Core concepts 4 and 5)

Project checks

- **Step 2.** The sample's plan:

Name	Inputs	Output and its type
`toFileName`	`pageName`	a string, like `about.html`
`makePageTitle`	`pageName`, `ownerName`	a string, like `About - Sam`
`isNarrowScreen`	`width`	a Boolean, `true` or `false`
`navLabel`	`fileName`	a string, like `About`, or `null`

- **Step 3.** Nothing prints, and there is no error.
- **Step 4.** Before the function: the heading line, then `ReferenceError: toFileName is not defined`, pointing at the first test line (line 10 in the sample). After it: `--- toFileName ---`, `index.html`, `about.html` and `contact.html`.
- **Step 5, continue alone.** The sample's `makePageTitle`:

```
// makePageTitle: takes a page name and your name and returns the text
// for the browser tab: the page name, a dash, then your name.
function makePageTitle(pageName, ownerName) {
  return pageName + " - " + ownerName;
}
```

- **Step 6, create alone.** The sample's tests and function. The tests go in the tests part, the function in the helpers part.

```
console.log("--- isNarrowScreen ---");
// expect true, false, false, true
console.log(isNarrowScreen(390));
console.log(isNarrowScreen(1200));
console.log(isNarrowScreen(700));
console.log(isNarrowScreen(699));
```

```
// isNarrowScreen: takes a screen width in pixels and returns true
// when it is narrower than 700, where the wider layout starts.
function isNarrowScreen(width) {
```


© 2026 559 Solutions LLC

```
  return width < 700;
}
```

- **Step 7, continue alone.** The sample's whole `navLabel`, with one `if` for each of its five pages:

```
// navLabel: takes a file name and returns the words for its nav link,
// or null when the site has no page with that file name.
function navLabel(fileName) {
  if (fileName === "index.html") {
    return "Home";
  }
  if (fileName === "about.html") {
    return "About";
  }
  if (fileName === "interests.html") {
    return "Interests";
  }
  if (fileName === "rate.html") {
    return "Rate";
  }
  if (fileName === "contact.html") {
    return "Contact";
  }
  return null;
}
```

- **Step 8.** Without the check, the last line is `null - Sam`. With the check, it is `blog.html is not a page on this site`. With `"about.html"` on the `const label` line, it is `About - Sam`.
- **Step 9.** With `<=`, only the test for 700 changes, from `false` to `true`. With the arguments swapped, the second title is `Sam - About`.
- **Step 10.** With `console.log()` in place of `return`, the `makePageTitle` group prints six lines: each title, then `undefined`. With no parentheses, the second `toFileName` line is `[Function: toFileName]`.
- **Step 11.** The `navLabel` group prints `Home`, `null`, `null`, `null`, `true` and `true`. The second and the last lines fail. A test with only `index.html` passes either way, because `"index.html"` matches the first `if` and returns `Home` before the `else` can matter. Only a file name that should reach a later `if` shows the bug.
- **Step 14.** If you followed the brief, every box is ticked. The most common misses: a `console.log()` left inside a function from step 10, and a test whose `// expect` comment was

© 2026 559 Solutions LLC

never filled in. Here is the whole sample file, 85 lines:

```
// helpers.js: four small helper functions for my site, with their tests
// Run it from the my-site folder with: node helpers.js

// ===== The helpers =====

// toFileName: takes a page name in lowercase and returns its file name,
// the name followed by .html, except home, which is index.html.
function toFileName(pageName) {
  if (pageName === "home") {
    return "index.html";
  }
  return pageName + ".html";
}

// makePageTitle: takes a page name and your name and returns the text
// for the browser tab: the page name, a dash, then your name.
function makePageTitle(pageName, ownerName) {
  return pageName + " - " + ownerName;
}

// isNarrowScreen: takes a screen width in pixels and returns true
// when it is narrower than 700, where the wider layout starts.
function isNarrowScreen(width) {
  return width < 700;
}

// navLabel: takes a file name and returns the words for its nav link,
// or null when the site has no page with that file name.
function navLabel(fileName) {
  if (fileName === "index.html") {
    return "Home";
  }
  if (fileName === "about.html") {
    return "About";
  }
  if (fileName === "interests.html") {
    return "Interests";
  }
  if (fileName === "rate.html") {
    return "Rate";
  }
  if (fileName === "contact.html") {
    return "Contact";
  }
  return null;
```


© 2026 559 Solutions LLC

```
}

// ===== The tests =====

console.log("--- toFileName ---");
// expect index.html, about.html, contact.html
console.log(toFileName("home"));
console.log(toFileName("about"));
console.log(toFileName("contact"));

console.log("--- makePageTitle ---");
// expect Home - Sam, About - Sam, Rate this lesson - Sam
console.log(makePageTitle("Home", "Sam"));
console.log(makePageTitle("About", "Sam"));
console.log(makePageTitle("Rate this lesson", "Sam"));

console.log("--- isNarrowScreen ---");
// expect true, false, false, true
console.log(isNarrowScreen(390));
console.log(isNarrowScreen(1200));
console.log(isNarrowScreen(700));
console.log(isNarrowScreen(699));

console.log("--- navLabel ---");
// expect Home, Contact, null, null, true, false
console.log(navLabel("index.html"));
console.log(navLabel("contact.html"));
console.log(navLabel("blog.html"));
console.log(navLabel("About.html"));
console.log(navLabel("blog.html") === null);
console.log(navLabel("about.html") === null);

// ===== Using two helpers together =====

const label = navLabel("blog.html");
if (label === null) {
  console.log("blog.html is not a page on this site");
} else {
  console.log(makePageTitle(label, "Sam"));
}
```

NEXT

Module 8: Arrays, Loops, and Objects.
Free lesson: 559solutions.com/code559/module-08-arrays-loops-objects/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC